

Kreacijski dizajn paterni

Singleton

Singleton patern osigurava da određena klasa ima samo jednu instancu tokom rada sistema te omogućava globalni pristup toj instanci. Ovaj patern se koristi kada je potrebno centralizovano upravljanje resursima ili konfiguracijama sistema.

U našem sistemu biblioteke Singleton bi se mogao primijeniti na klasu BibliotekaSystem. Sistem biblioteke treba imati jedinstvenu centralnu instancu koja upravlja kolekcijama knjiga, korisnika, rezervacija i posudbi. Kreiranjem privatnog konstruktora i statičke metode getInstance(), osigurava se da tokom rada aplikacije postoji samo jedna instanca sistema biblioteke. Na taj način svi dijelovi aplikacije koriste isti skup podataka i izbjegavaju se problemi nekonzistentnosti.

Factory Method

Factory Method patern definiše interfejs za kreiranje objekata, dok konkretne podklase odlučuju koji će se objekat zapravo instancirati. Time se logika kreiranja objekata odvaja od ostatka aplikacije i olakšava proširivanje sistema novim tipovima objekata.

U našem sistemu biblioteke ovaj patern bi se mogao primijeniti pri kreiranju korisnika različitih uloga. Sistem sadrži više tipova korisnika pri čemu svaki ima različita prava i funkcionalnosti. Implementacijom klase KorisnikFactory sa metodom kreirajKorisnika(), sistem bi na osnovu prosljeđene uloge automatski kreirao odgovarajući tip korisnika. Na taj način bi se centralizovala logika kreiranja objekata i pojednostavilo dodavanje novih korisničkih uloga u budućnosti.

Abstract Factory

Abstract Factory patern omogućava kreiranje porodica povezanih objekata bez specificiranja njihovih konkretnih klasa. Ovaj patern je naročito koristan kada sistem podržava više međusobno povezanih konfiguracija ili modova rada.

U okviru našeg bibliotečkog sistema ovaj patern bi se mogao koristiti za kreiranje različitih tipova obavještenja i komunikacijskih servisa. Na primjer, jedna konkretna fabrika mogla bi kreirati EmailObavjestenje i EmailServis, dok bi druga kreirala SMSObavjestenje i SMSServis. Klijentski kod bi koristio zajednički interfejs fabrike bez potrebe da zna konkretne implementacije. Time bi

se omogućilo jednostavno dodavanje novih kanala komunikacije bez izmjene postojećeg sistema.

Builder

Builder patern služi za postepenu konstrukciju složenih objekata. Umjesto velikog konstruktora sa mnogo parametara, objekat se gradi korak po korak, što povećava čitljivost i fleksibilnost koda.

U našem sistemu biblioteke Builder bi se mogao koristiti za kreiranje objekta Knjiga. Klasa Knjiga sadrži veliki broj atributa poput naslova, autora, ISBN broja, jezika, žanra, opisa, broja stranica i godine izdavanja. Korištenjem klase KnjigaBuilder omogućilo bi se postepeno definisanje svih potrebnih podataka prije konačnog kreiranja objekta. Na ovaj način bi se izbjegli predugi konstruktori i povećala preglednost koda prilikom dodavanja novih knjiga u sistem.

Prototype

Prototype patern omogućava kreiranje novih objekata kopiranjem postojećih instanci umjesto njihovim ponovnim kreiranjem od početka. Ovaj pristup je koristan kada je kreiranje objekta skupo ili kompleksno.

U sistemu biblioteke ovaj patern bi se mogao primijeniti na klasu Knjiga ili Primjerak. Kada biblioteka nabavlja više primjeraka iste knjige, novi primjerci bi se mogli kreirati kloniranjem postojećeg objekta uz minimalne izmjene poput inventarskog broja ili statusa dostupnosti. Na taj način bi se ubrzao proces kreiranja novih objekata i smanjilo ponavljanje istih podataka.

Object Pool

Object Pool patern upravlja skupom unaprijed kreiranih objekata koji se ponovo koriste umjesto da se stalno kreiraju i uništavaju. Ovaj patern je koristan kod objekata čije je kreiranje skupo po performanse sistema.

U našem sistemu biblioteke ovaj patern bi se mogao koristiti za upravljanje konekcijama prema bazi podataka ili servisima za slanje obavještenja. Umjesto da se za svaki zahtjev kreira nova konekcija, sistem bi održavao skup već otvorenih konekcija koje bi se dodjeljivale po potrebi. Nakon završetka rada konekcija bi se vraćala u pool za narednu upotrebu. Na taj način bi se poboljšale performanse sistema i smanjilo opterećenje resursa.